

DATA 260 - HOMEWORK 5

Restaurant Inspection API, MCP Tools, Reliability, and Agent Loop

Student / Author: Bhushan Waingankar
Repository: <https://github.com/shoddy16/data260-7085>
Domain: Local restaurant inspections (DOMAIN_ID 5)

Value	Configured value
SID4	7085
PORT_BASE	8785
PREFIX	s7085
SEED	7085
VERIFY_SEED	267085
DOMAIN_ID	5 - Local restaurant inspections
Hardware	Windows 10; Intel Core i5-6200U at 2.30 GHz; 4 logical CPUs; approximately 7.8 GB RAM
Local model	Ollama llama3.2:3b at http://localhost:11434
Tag / commit	Pending evidence capture and final Git commit. Base HEAD: fb6884388a43142bf983566125e4fc0070ddf557

DRAFT FOR EVIDENCE COMPLETION. Yellow panels identify required GUI screenshots not yet saved; they are placeholders, not evidence.

The repository URL is included. Access for collaborators Sbnikitha and supriyaselvangesan was not verified in this environment; confirm access before submission.

1. Database and API

The HW4 FastAPI/MySQL service now includes a related Restaurant entity while retaining Inspection records. The additive migration preserved existing records, backfilled the relationship, and added constraints and timestamps.

Relationship and schema

Source excerpt - models/models.py

```
class Restaurant(Base):
    __tablename__ = "restaurants"
    __table_args__ = (UniqueConstraint("code", name="uq_restaurants_code"),)
    id = Column(Integer, primary_key=True, index=True)
    name = Column(String(255), nullable=False, index=True)
    location = Column(String(500), nullable=False)
    code = Column(String(32), nullable=False, index=True)
    inspections = relationship("Inspection", back_populates="restaurant")

class Inspection(Base):
    restaurant_id = Column(Integer, ForeignKey("restaurants.id", ondelete="RESTRICT"), nullable=False)
    inspection_code = Column(String(64), nullable=False, unique=True)
    score = Column(Integer, nullable=False, default=100)
    restaurant = relationship("Restaurant", back_populates="inspections")
```

The restrictive foreign key prevents deleting a restaurant while related inspections exist. The migration verification found 5,000 inspections linked to 5,001 restaurants, including the original Harbor Cafe row; no required fields were null and inspection codes were unique.

Read-only check	Observed result
Inspection data	5,000 rows; no null links or required HW5 values
Restaurant data	5,001 rows, including the pre-existing Harbor Cafe
Foreign key	inspections.restaurant_id -> restaurants.id; ON DELETE RESTRICT
Paginated API	GET /restaurants/?skip=0&limit;=1 and GET /inspections/?skip=0&limit;=1 returned HTTP 200

MANUAL SCREENSHOT REQUIRED

P1-01-database-schema-and-rows.png

MySQL GUI showing tables, constraints, counts, and linked sample rows. Hide credentials.

1.1 API endpoints, CRUD, and validation

Both entities have create, paginated list, read-one, update, and delete endpoints. A relationship route returns inspections for a restaurant. Pydantic validation and CRUD handling return appropriate 404, 409, and 422 responses.

Source excerpt - routers/restaurants.py

```
@router.get("/", response_model=list[RestaurantResponse])
def read_all(
    skip: int = Query(0, ge=0),
    limit: int = Query(50, ge=1, le=200),
    db: Session = Depends(get_db),
):
    return get_restaurants(db, skip=skip, limit=limit)

@router.get("/{restaurant_id}/inspections", response_model=list[InspectionResponse])
def read_inspections(
    restaurant_id: int,
    skip: int = Query(0, ge=0),
    limit: int = Query(50, ge=1, le=200),
    db: Session = Depends(get_db),
):
    if not get_restaurant(db, restaurant_id):
        raise HTTPException(status_code=404, detail="Restaurant not found")
    return get_inspections_by_restaurant(db, restaurant_id, skip=skip, limit=limit)
```

Automated API result: the post-change Python suite passed 24 tests. Actual test cases include HTTP 201 create, 200 reads/updates, 204 deletes, 404 missing resources, 409 duplicate/restrictive-delete conflicts, and 422 validation errors.

Postman screenshots - each action is a separate labeled capture. Use only newly created temporary evidence records and delete the child before its parent.

MANUAL SCREENSHOT REQUIRED P1-02-restaurant-create.png POST /restaurants/ - 201	MANUAL SCREENSHOT REQUIRED P1-03-restaurant-list-pagination.png GET /restaurants/?skip=0&limit=5
MANUAL SCREENSHOT REQUIRED P1-04-restaurant-detail.png GET /restaurants/{new_id}	MANUAL SCREENSHOT REQUIRED P1-05-restaurant-update.png PUT /restaurants/{new_id} - updated record
MANUAL SCREENSHOT REQUIRED P1-06-restaurant-delete.png DELETE the new parent after removing its child	MANUAL SCREENSHOT REQUIRED P1-07-inspection-create.png POST /inspections/ - linked child
MANUAL SCREENSHOT REQUIRED P1-08-inspection-list-pagination.png GET /inspections/?skip=0&limit=5	MANUAL SCREENSHOT REQUIRED P1-09-inspection-detail.png GET /inspections/{new_id}
MANUAL SCREENSHOT REQUIRED P1-10-inspection-update.png PUT /inspections/{new_id} - score update	MANUAL SCREENSHOT REQUIRED P1-11-inspection-delete.png DELETE the new inspection
MANUAL SCREENSHOT REQUIRED P1-12-relationship.png GET /restaurants/{new_id}/inspections	MANUAL SCREENSHOT REQUIRED P1-13-404.png GET /restaurants/99999 - 404
MANUAL SCREENSHOT REQUIRED P1-14-409.png Delete parent while child exists - 409	MANUAL SCREENSHOT REQUIRED P1-15-validation.png POST with malformed restaurant code - 422

1.2 Redux Toolkit client

Axios and Redux Toolkit async thunks connect React screens to FastAPI. Slice reducers update state after successful create, update, and delete operations; errors are returned to the UI.

Source excerpt - frontend/src/features/inspections/inspectionsSlice.js

```
export const fetchInspections = createAsyncThunk(
  "inspections/fetchAll",
  async (_, { rejectWithValue }) => {
    try { return (await api.get("/inspections/")).data; }
    catch (error) { return rejectWithValue(errorMessage(error, "Could not load inspections.")); }
  },
);

.addCase(createInspection.fulfilled, (state, action) => { state.items.unshift(action.payload); })
.addCase(updateInspection.fulfilled, (state, action) => {
  const index = state.items.findIndex((item) => item.id === action.payload.id);
  if (index !== -1) state.items[index] = action.payload;
})
.addCase(deleteInspection.fulfilled, (state, action) => {
  state.items = state.items.filter((item) => item.id !== action.payload);
});
```

The frontend production build passed with 96 modules transformed. The assignment requires each UI screenshot to pair the relevant Redux code and corresponding browser result in one frame.

MANUAL SCREENSHOT REQUIRED P1-16-redux-home.png Home/list UI plus list thunk or slice	MANUAL SCREENSHOT REQUIRED P1-17-redux-create.png Create form/result plus create thunk or slice
MANUAL SCREENSHOT REQUIRED P1-18-redux-update.png Update flow/result plus update thunk or slice	MANUAL SCREENSHOT REQUIRED P1-19-redux-delete.png Delete flow/list result plus delete thunk or slice

2. MCP servers - TheMealDB

The local `meals` MCP server exposes four tools. Real calls to the public TheMealDB service completed; actual JSON is preserved in raw/mealdb_live_results.json.

Source excerpt - meals_server.py

```
@mcp.tool()
async def search_meals_by_name(query: str, limit: int = 5) -> dict:
    return await mealdb.search_meals_by_name(query, limit)

@mcp.tool()
async def meals_by_ingredient(ingredient: str, limit: int = 12) -> dict:
    return await mealdb.meals_by_ingredient(ingredient, limit)

@mcp.tool()
async def random_meal() -> dict:
    return await mealdb.random_meal()

@mcp.tool()
async def meal_details(id: str | int) -> dict:
    return await mealdb.meal_details(id)
```

Tool	Actual live result	Returned fields
search_meals_by_name	Spicy Arrabiata Penne (id 52771)	id, name, area, category, thumb
meals_by_ingredient	Brown Stew Chicken, Chicken & chorizo rice pot	id, name, thumb
random_meal	Fish fofos (id 53043)	full recipe, instructions, image, source, video, ingredients
meal_details	Teriyaki Chicken Casserole (id 52772)	full recipe, instructions, image, source, video, ingredients

Inspector screenshots - show the selected tool, exact input, and real returned output.

MANUAL SCREENSHOT REQUIRED P2-01-meal-search.png Search name Arrabiata; limit 2	MANUAL SCREENSHOT REQUIRED P2-02-meal-ingredient.png Ingredient chicken; limit 2
MANUAL SCREENSHOT REQUIRED P2-03-meal-random.png Random meal; no input	MANUAL SCREENSHOT REQUIRED P2-04-meal-details.png Meal id 52772

2.1 MCP server - restaurant domain

The domain server declares exactly `search`, `detail\_lookup`, and `aggregate`. All tools use the same `{ok, data, error}` envelope. MCP Inspector was used for valid and deliberately invalid calls; JSON receipts are preserved under raw/.

Source excerpt - domain_mcp_server.py

```
@mcp.tool()
def search(query: str, limit: int = 5) -> dict:
    return default_inspection_tools.search(query, limit)

@mcp.tool()
def detail_lookup(inspection_id: int) -> dict:
    return default_inspection_tools.detail_lookup(inspection_id)

@mcp.tool()
def aggregate(group_by: str = "category") -> dict:
    return default_inspection_tools.aggregate(group_by)
```

Tool / expected input	Rejected input	Returned error envelope	Reason
search: {'query': 'string (non-empty)', 'limit': 'integer 1..25; optional, default 5'}	{ "query": " ", "limit": 5 }	{ "ok": false, "data": null, "error": "query must be a non-empty string" }	The query contains only whitespace.
detail_lookup: {'inspection_id': 'positive integer'}	{ "inspection_id" : 0 }	{ "ok": false, "data": null, "error": "inspection_id must be a positive integer" }	Zero is not a valid positive inspection identifier.
aggregate: {'group_by': 'category or restaurant; optional, default category'}	{ "group_by": "email" }	{ "ok": false, "data": null, "error": "group_by must be 'category' or 'restaurant'" }	Email is not an allowed aggregate dimension.

Live category aggregation returned 1,686 Food Safety, 1,683 Health and Safety, and 1,631 Sanitation records. A detail call returned inspection id 4 for Restaurant 1. The rejected-call JSON preserves the error outputs and explains why each input is invalid.

Inspector screenshots - one success and one invalid call for each domain tool.

MANUAL SCREENSHOT REQUIRED P2-05-domain-search-ok.png search: Restaurant 1, limit 2	MANUAL SCREENSHOT REQUIRED P2-06-domain-search-invalid.png search: blank query
MANUAL SCREENSHOT REQUIRED P2-07-domain-detail-ok.png detail_lookup: id 4	MANUAL SCREENSHOT REQUIRED P2-08-domain-detail-invalid.png detail_lookup: id 0
MANUAL SCREENSHOT REQUIRED P2-09-domain-aggregate-ok.png aggregate: group_by category	MANUAL SCREENSHOT REQUIRED P2-10-domain-aggregate-invalid.png aggregate: group_by email

3. Tool contracts under stress

External API and read operations use timeouts and bounded exponential backoff. A seeded fault injector ran 50 calls at each required failure rate; the full 150-record CSV/JSONL dataset is preserved.

Source excerpt - reliability.py

```
def _backoff_seconds(failed_attempt, base, maximum):
    return min(base * (2**failed_attempt), maximum)

for attempt in range(max_attempts):
    try:
        return operation()
    except Exception as error:
        if attempt + 1 >= max_attempts or not retryable(error):
            raise
        sleep(_backoff_seconds(attempt, base_backoff_seconds, max_backoff_seconds))
```

Injected failure	Calls	Success	Mean latency	p99 latency
0%	50	100.0%	1.6720 ms	4.1288 ms
20%	50	100.0%	6.0630 ms	37.4028 ms
50%	50	90.0%	13.6744 ms	37.1493 ms

Policy: three attempts; 50 ms base backoff; 200 ms cap; 1 ms simulated operation work. At 50% injected failure, five of fifty calls failed. This bounded policy is useful for occasional interactive failures; batch jobs can use a larger retry budget and longer delay cap while retaining per-item failures.

Demonstration	Attempts	Actual outcome
First-attempt success	1	ok
Retry then success	2	ok
Retry exhaustion	3	all attempts failed

MANUAL SCREENSHOT REQUIRED
P3-01-fault-injection-run.png
Terminal: 150 calls, 50 at each rate

MANUAL SCREENSHOT REQUIRED
P3-02-retry-examples.png
Actual first success, retry success, exhaustion

MANUAL SCREENSHOT REQUIRED
P3-03-metrics.png
Measured success, mean, and p99

4. One safe tool entry point and offline tests

``execute_tool()`` is the single domain-tool dispatcher for the agent. It validates inputs, applies the restaurant-inspection safety rule, catches errors, and serializes the common result envelope.

Source excerpt - `tool_executor.py`

```
if _is_safety_blocked(name, inputs):
    return json.dumps(failure("safety rule blocked requests to evade or falsify restaurant inspections"))

handlers = {
    "search": lambda: domain_tools.search(inputs.get("query"), inputs.get("limit", 5)),
    "detail_lookup": lambda: domain_tools.detail_lookup(inputs.get("inspection_id")),
    "aggregate": lambda: domain_tools.aggregate(inputs.get("group_by", "category")),
}
try:
    envelope = handler()
except Exception:
    envelope = failure("tool execution failed")
return json.dumps(envelope, ensure_ascii=False, separators=(",", ":"))
```

Actual fixture examples

`raw/safety_examples.json`

```
{
  "allowed": {
    "ok": true,
    "data": [
      {
        "id": 3,
        "restaurant": "Juniper",
        "category": "Routine"
      }
    ],
    "error": null
  },
  "blocked": {
    "ok": false,
    "data": null,
    "error": "safety rule blocked requests to evade or falsify restaurant inspections"
  }
}
```

The allowed Harbor search returned one fixture record with ``ok: true``. The prohibited request returned ``ok: false``, ``data: null``, and a clear safety error without raising an exception.

4.1 Offline test results

The post-change Python suite passed 24 tests. The offline assert runner passed all nine checks without a live database, Internet API, or model.

Actual console output - run_offline_tests.py	
<pre>PASS: search valid PASS: search invalid blank query PASS: detail valid PASS: detail invalid id PASS: aggregate valid PASS: aggregate invalid group PASS: safety rule blocks evasion PASS: MockModel stops at max_steps PASS: MealDB shape and limit (mocked) OFFLINE ASSERTIONS: 9/9 passed</pre>	
Actual MockModel result - raw/max_steps_example.json	
<pre>{ "answer": "The agent stopped because it reached the configured max_steps limit.", "step_count": 2, "tool_call_count": 2, "stop_reason": "max_steps" }</pre>	
MANUAL SCREENSHOT REQUIRED P4-01-offline-tests.png Terminal with every PASS and final 9/9	MANUAL SCREENSHOT REQUIRED P4-02-safety-allowed.png Allowed fixture envelope
MANUAL SCREENSHOT REQUIRED P4-03-safety-blocked.png Blocked safety envelope	MANUAL SCREENSHOT REQUIRED P4-04-max-steps.png MockModel reaching max_steps

5. Agent loop, safety, and metrics

`run_agent()` asks the local model for a structured action, routes tool calls through `execute_tool()`, inspects results, records each step, and stops at `max_steps`. The required model remained llama3.2:3b.

Source excerpt - agent.py

```
while step_count < max_steps:
    step_count += 1
    action = agent_model.next_action(messages)
    # Validate the structured action and dispatch through execute_tool.
    result_text = tool_executor(tool_name, tool_inputs)
    result = json.loads(result_text)
    # Add the actual result to the next model turn and append a JSONL step.
    if step_count >= max_steps:
        stop_reason = "max_steps"
        break
```

Scenario	Tool	Steps	Tool calls	Stop reason
Search Restaurant 1 inspections in Mountain View	search	2	1	normal_completion
Show details for inspection 4	detail_lookup	2	1	normal_completion
Count inspections by category	aggregate	2	1	normal_completion
Find inspections in the Sanitation category	aggregate	2	1	normal_completion

All four focused real Ollama scenarios completed normally in two model steps with one tool call. Their run IDs, answers, and tool names are in `raw/agent_scenario_summary.json`; full step events are in `raw/agent_runs.jsonl`. An oversized per-restaurant aggregate timed out and remains recorded separately; it is not described as a success.

MANUAL SCREENSHOT REQUIRED

P5-01-agent-scenarios.png

Terminal with four prompts, steps, stop reasons, and tool counts

MANUAL SCREENSHOT REQUIRED

P5-02-agent-log.png

Actual JSONL step and run completion records

5.1 Reflection

The following reflection is based on the actual successful Ollama search run recorded in `raw/agent_runs.jsonl`.

This reflection is based on the real Ollama run ``f1a4d06c-de38-4919-9dee-4b3a3683d879`` in ``raw/agent_runs.jsonl``. I asked the agent to find inspections for Restaurant 1 in Mountain View. At its first step, the configured ``llama3.2:3b`` model selected the ``search`` tool with the restaurant/location query and a limit of five. The agent passed that request through ``execute_tool()``; it did not invoke the repository directly or bypass the common result envelope. The tool returned a successful result containing the matching inspection record, including inspection ID 4, code ``s7085-i-00000004``, score 100, category, location, and description.

At the second step, the model used the returned tool data and produced a final user-facing answer. The run ended with ``normal_completion``, two model steps, and one tool call. This run followed an earlier failed search trace that had matched restaurant numbers by substring, returning Restaurant 11 and Restaurant 19 along with Restaurant 1. The earlier output made the relevance defect visible. The search implementation was tightened to match numeric query tokens as whole words across restaurant and inspection fields, and a direct MySQL query then returned only Restaurant 1 for the same request. The rerun's saved response described the exact matching record.

The run demonstrates the intended loop: the model selects a domain tool, the safe dispatcher returns data, and the model answers from that result. The bounded loop remains necessary: another actual request for all per-restaurant aggregates produced a large result and timed out on a later model turn. That unsuccessful trace is retained rather than removed, and the selected four focused scenario metrics are reported separately.

The reflection is 200-300 words as required.

6. AI Use

AI-use disclosure included in the report as required by the assignment.

1. ChatGPT/Codex assisted with repository inspection, implementation review, tests, MCP and agent code, documentation, and this report. The student supplied the HW5 PDF, Professor Demo 5 ZIP, and existing repository and remains responsible for reviewing the work and capturing required UI evidence.
2. An early implementation/search assumption was incorrect: substring matching treated restaurant number `1` as matching `11` and `19`. Early real Ollama traces also showed repeated calls and unsupported aggregate groupings.
3. These issues were identified from actual MySQL tool results and the saved JSONL run trace. The search now applies whole-token matching for numeric terms, and the agent prompt now explicitly directs the model to answer after a successful result. The same live search now returns only Restaurant 1; four focused real Ollama scenarios completed normally. Failed/time-out traces remain in the raw log.
4. The additive MySQL migration was reviewed and run without a destructive reset after preserving an export of affected table rows. It was verified against the actual schema and records. Python tests, offline assertions, frontend build, MCP calls, and model scenarios were run; current verification status is tracked in `RUN\_LOG.txt` and `verification.json`. Required Postman/Redux/database/Inspector screenshots still require manual capture in the desktop UI.

7. Verification and remaining evidence

The consolidated script passed 11 checks on the working tree, including the Python suite, offline assertions, Vite build, read-only MySQL/API checks, saved MCP and MealDB results, four normal real Ollama runs, and the 150-record experiment dataset.

Verification check	Result
Python unittest suite	PASS
Offline assertion runner	PASS
Frontend production build	PASS
Meals server declares four required tools	PASS
Domain server declares exactly three tools	PASS
Configured MySQL contains migrated HW5 schema/data	PASS
Live MySQL paginated API GET endpoints	PASS
Saved live TheMealDB calls cover four tools	PASS
Saved live MCP stdio discovery for both servers	PASS
Four saved real Ollama scenarios completed normally	PASS
Saved fault-injection raw data has 150 calls	PASS

This remains a draft. Zero GUI screenshots have been saved; collaborator access has not been confirmed; the final submission PDF, commit, and `hw5` tag are still pending. After saving the named screenshots under `reports/hw05/screenshots/`, rerun this builder to replace each yellow placeholder with the actual image, visually inspect the updated PDF, refresh verification, then commit and tag the finished state.